Week 7: Diffusion and flow matching

Jin-Guo Liu, Xi-Wei Pan

Abstract: We learn to generate data from Gaussian noise, starting with four groups of points, called clusters, whose distribution we can check exactly. We train a noise predictor, compare sampling methods, then use a downloaded predictor to generate photographs and fill missing pixels.

Diffusion models

Generating data from Gaussian noise. Week 6 generated samples from an energy model without computing its normalizer Z. This week learns to transform fresh random numbers into data. A *generative model* learns a rule for drawing new examples; drawing an example is *sampling*, and the result is a *sample*. We use a *neural network*, a function with adjustable parameters. *Training* fits those parameters to examples.

Gaussian noise as the starting distribution. We start generation with independent draws from the *standard Gaussian* $\mathcal{N}(0, 1)$, centred at zero with *variance* one: its average squared distance from zero is one. We draw coordinates *independently*, so knowing one draw does not change another's probabilities. Added random disturbances are noise a *clean sample* x_0 is a data sample before this noise is added. Its coordinates may store a point's position or an image's pixel colours.

Forward and reverse diffusion. The fixed process called forward diffusion repeatedly shrinks x_0 toward zero and adds Gaussian noise. Write the current *state* as x_t after t steps. The step index is the noise level, also called the diffusion level here the final level is $T = 1000$. In Figure 1, read the top row left to right to see data become noise. Generation follows the *reverse process*, reading the lower rows right to left.

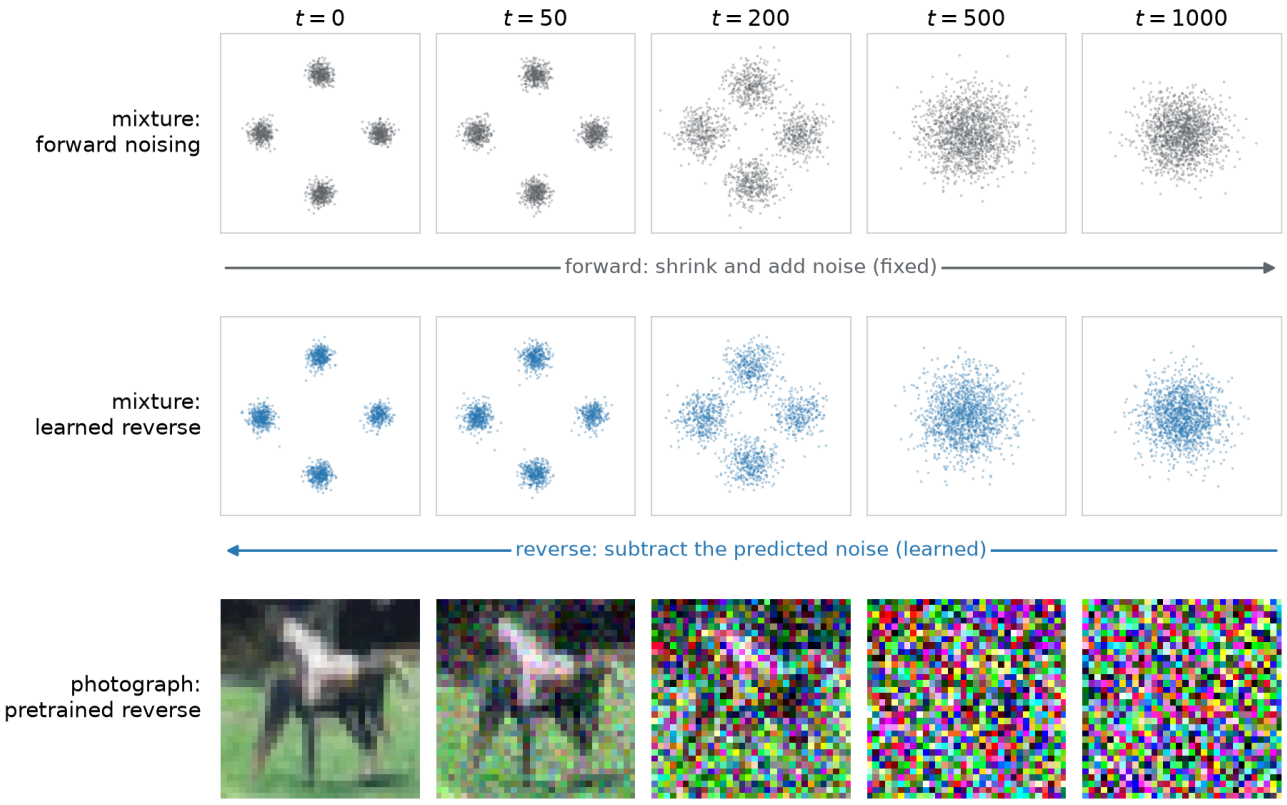


Figure 1: The forward and reverse processes. Top: forward diffusion turns four clusters into an approximately Gaussian cloud. Middle: a learned reverse process turns fresh Gaussian noise back into four clusters. Bottom: a downloaded network guides the reverse process for a 32×32 photograph. Columns use the noise levels defined above. All panels are reference runs from Parts 1 and 4.

Generation by small reverse steps. Starting from fresh Gaussian noise at $t = T$, a learned rule repeatedly moves to a lower level until it reaches $t = 0$. The positions form a *path* or *trajectory*; the first and last are its *endpoints*. The aim

is to recover the data distribution: which outcomes are common and which are rare. It does not recover the particular sample that began a forward run. Estimating a clean sample from a noisy one is *denoising*.

Noise prediction and DDPM sampling. Add noise to a clean sample and train a noise predictor to estimate the added noise. The known answer is the training *target*; fitting numerical targets is *regression*. A *denoising diffusion probabilistic model* (DDPM) uses the prediction to choose a reverse step and adds fresh randomness. This generation rule is the *sampler*.

Two routes to fewer sampling steps. Each reverse step evaluates a network. Part 2 reuses the noise predictor with a *denoising diffusion implicit model* (DDIM). Its deterministic update makes no random choice after the starting draw. DDPM's random steps are called *stochastic*. Part 3 trains a different network by flow matching: it predicts a point's *velocity*, how fast and in which direction the point moves. We compare the number of steps these samplers need to recover the four clusters.

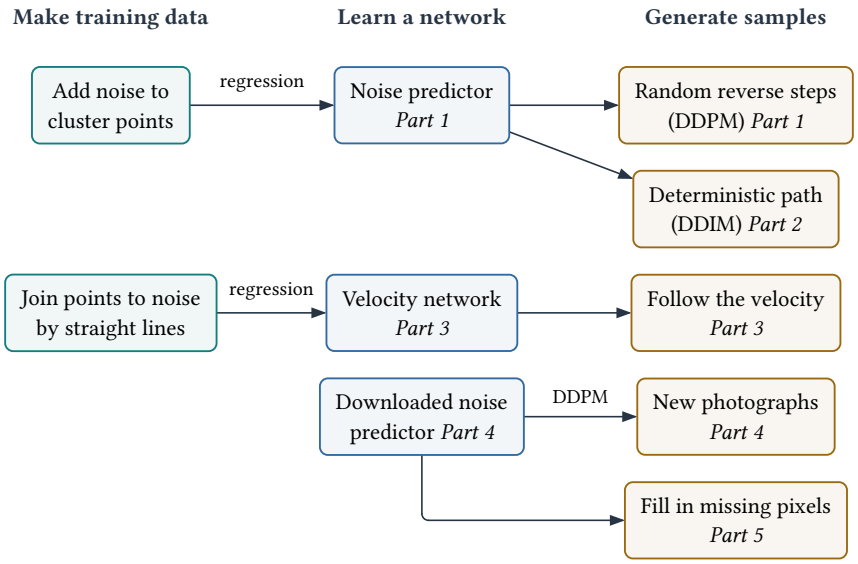


Figure 2: Which network each Part trains or reuses, and which sampler uses it. Parts 1 to 3 use the four-cluster test distribution; Parts 4 and 5 use photographs.

Resources:

- Niels Rogge and Kashif Rasul, *The Annotated Diffusion Model*. Parts 1 and 4 follow its noising, noise-prediction and reverse-sampling sequence. Our implementation uses JAX.
- Ho, Jain and Abbeel, *Denoising Diffusion Probabilistic Models*. Algorithms 1–2 are the training and sampling loops of Parts 1 and 4.

Part 1: Diffusion models on a Gaussian mixture

Goal: train a noise predictor and use it to generate new samples. Check that the samples form the four target clusters, with the right fraction of points in each cluster and the right centres and widths.

① DO

Work in `week7/` inside your existing exercise repository. Your laptop's central processing unit (CPU) is the baseline. Send the prompts, then read UNDERSTAND while your agent works.

```
Work in week7/ using uv, with CPU as the default. Save figures in evidence/ and raw
runs with settings in artifacts/. Keep downloads and raw runs out of git. Commit after
each task; ask about uncertainties instead of guessing.
```

```
Build forward diffusion in JAX for an equally weighted Gaussian mixture with centres
(3,0), (0,3), (-3,0), (0,-3) and covariance 0.09 times the identity. Use 1000 steps
with per-step noise variance beta_t increasing linearly from 0.0001 to 0.02, where t
is the diffusion level. Implement DDPM sampling with variance beta_t, no clipping and
no added noise in the final step.
```

```
Train a small network to predict the added noise. Make the predictor time-conditioned.
Minimize mean squared noise-prediction error over uniformly sampled noise levels.
Decay the learning rate to zero by the end of training. Record training settings and
loss history. Retain the model at 10 training stages, including initialization and the
final model.
```

For reference, 25000 updates with 512 fresh points per update take about one minute on a laptop CPU.

2 UNDERSTAND

Drawing the four-cluster target. Our sample is a pair of real numbers, a *vector* $x \in \mathbb{R}^2$. Choosing among Gaussian clusters gives a Gaussian mixture. Each cluster is a *component*. Our four centres are

$$\mu_1 = (3, 0), \quad \mu_2 = (0, 3), \quad \mu_3 = (-3, 0), \quad \mu_4 = (0, -3). \tag{1}$$

Give each component index $k \in \{1, 2, 3, 4\}$ equal probability and draw independent Gaussian numbers z_1, z_2, independently of k. Then

$$x_0 = \mu_k + 0.3(z_1, z_2), \quad z_1, z_2 \sim \mathcal{N}(0, 1) \ \text{ independently}. \tag{2}$$

These are *exact samples*, drawn directly from the prescribed target. Parts 1 to 3 use them to train and to check learned samplers.

Describing the target with a density and averages. A *probability density* $p(x)$ integrates to a region's probability and to one over the whole space. Its value at one point is not a probability. An *expected value* is a density-weighted average:

$$\mathbb{E}[h(x)] = \int h(x)p(x)\,\mathrm{d}x. \tag{3}$$

A sample average estimates this integral. The mean $\mu = \mathbb{E}[x]$ averages each coordinate separately.

Covariance and cluster width. The *covariance matrix* records the spread:

$$\Sigma_{ij} = \mathbb{E}\big[(x_i - \mu_i)(x_j - \mu_j)\big]. \tag{4}$$

Its diagonal entries are coordinate variances; off-diagonal entries describe how coordinates vary together. The square root of a variance is the *standard deviation*. Each component above has independent coordinates of standard deviation $\sigma = 0.3$, hence covariance $0.09I$. The *identity matrix* I has ones on its diagonal and zeros elsewhere.

The target density. Write $\mathcal{N}(x; \mu, \Sigma)$ for the Gaussian density with mean μ and covariance Σ. The recipe gives

$$p_{\text{data}}(x) = \frac{1}{4}\sum_{k=1}^{4} \mathcal{N}(x; \mu_k, 0.09I). \tag{5}$$

Its four separated regions of high density are the target's modes. The notation $z \sim \mathcal{N}(0, I)$ will mean a vector of independent standard Gaussian draws, each with mean zero and variance one.

Checking cluster populations, centres and widths. Assign generated points to their nearest target centre. Each group's *population*, its fraction of all points, should be near 0.25. Its mean should match μ_k, and its *per-coordinate spread*, the standard deviation in each coordinate, should be near 0.3. Checking all three detects missing, misplaced and broadened clusters.

Adding noise with the prescribed variance schedule. The forward update shrinks a point and adds independent noise. The diffusion level runs through $t \in \{1, \ldots, T\}$, with $T = 1000$. Let β_t be the variance of the noise added to each coordinate in step t, and set $\alpha_t = 1 - \beta_t$. The update is

$$x_t = \sqrt{\alpha_t}\,x_{t-1} + \sqrt{\beta_t}\,z, \quad z \sim \mathcal{N}(0, I). \tag{6}$$

The *variance schedule* $\beta_1, \ldots, \beta_T$ grows linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$, so early steps add less noise than later ones. The shrinkage multiplies the previous variance by α_t; independent noise adds β_t. Since $\alpha_t + \beta_t = 1$, variance one stays one.

Direct noising to any level. To construct a training example at level t, combine the preceding Gaussian disturbances into one draw. Define $\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\overline{\alpha}_0 = 1$. The result of the t steps has the same distribution as

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \tag{7}$$

Here ε is the known noise for this training example. At $t = 1000$, $\overline{\alpha}_t \approx 4 \times 10^{-5}$, so the clean point contributes very little and the state is approximately standard Gaussian.

Predicting noise from the point and its level. The input is (x_t, t) and the output $\varepsilon_\theta(x_t, t)$ has two coordinates, one estimated noise value per coordinate of the point. The parameters are θ. Supplying the noise level makes the network time-conditioned the index t is also called *time*. The clean point x_0 is unavailable during generation and is never an input.

The predictor's architecture and weights. The reference network computes fixed sine and cosine functions of the level t. Collect these *time features* in a vector $r(t)$ and append them to the two coordinates of x_t, giving $h_0 = [x_t; r(t)]$. The semicolon means joining the two lists of numbers. Three *hidden layers*, intermediate vectors h_1, h_2, h_3, then compute

$$h_l = \varphi(W_l h_{l-1} + b_l), \quad l = 1, 2, 3, \quad \varepsilon_\theta(x_t, t) = W_4 h_3 + b_4. \tag{8}$$

Each matrix W_l maps the preceding vector to the next layer's size, and b_l is an added vector, called a *bias*. Every output of this matrix multiplication can depend on every input, so the layer is *fully connected*. The function $\varphi(u) = \frac{u}{1+\exp(-u)}$, applied to each coordinate, is the *activation*; its nonlinear response prevents the layers from collapsing into one matrix multiplication. The final layer returns two noise coordinates without an activation.

This arrangement is the network's architecture. Its adjustable weights, or *parameters*, are $\theta = \{W_l, b_l\}_{l=1}^{4}$. *Initialization* assigns their starting values; the reference uses random matrices and zero biases. Training changes θ while keeping the time features and architecture fixed.

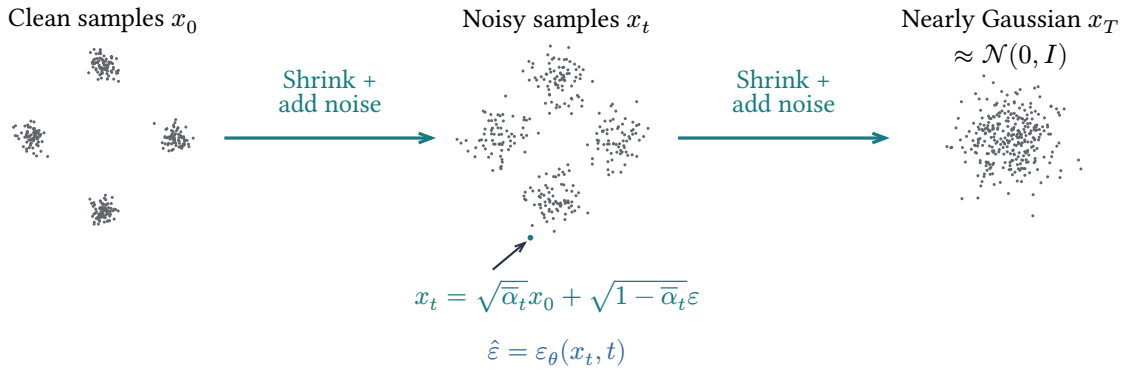


Figure 3: Training inputs at three noise levels, using the same 300 reference samples. The arrow follows one sample. Teal constructs the noisy input from known noise; blue predicts that noise from the input and its level.

The noise-prediction loss. To judge a prediction, subtract the known noise and square each coordinate's error. For a vector w, its length, or *norm*, is $\|w\| = \sqrt{\sum_i w_i^2}$. Thus $\|w\|^2$ is the sum of its squared coordinates. The mean squared noise-prediction error is

$$\mathcal{L}_{\text{noise}}(\theta) = \mathbb{E}_{x_0, t, \varepsilon}\left[\frac{1}{2}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2\right]. \tag{9}$$

The expectation averages over clean points, noise levels and Gaussian noise. We draw levels *uniformly*, giving every level the same probability. The factor $\frac{1}{2}$ averages over the two coordinates. This quantity is the training *loss*, the number we try to reduce.

Updating the weights during training. A *batch* is a group of training examples. Its average loss is $\hat{\mathcal{L}}$. The *gradient* $\nabla_\theta \hat{\mathcal{L}}$ collects the derivatives with respect to the weights; JAX computes it by automatic differentiation, as in Week 5. An *optimizer* uses this gradient to update the weights. For example, gradient descent uses $\theta \leftarrow \theta - \eta_j \nabla_\theta \hat{\mathcal{L}}$, where the *learning rate* η_j sets the step size at update j. Decaying η_j toward zero makes late updates smaller. A saved set of weights is a *training stage*.

Algorithm 1. Train the noise predictor

Input: Initial weights θ; batch size B; J updates with learning rates η_j; optimizer; data distribution p_{data}; noise schedule $\overline{\alpha}_t$.
Output: Trained weights θ.

for $j = 1, \ldots, J$ **do**
  for $i = 1, \ldots, B$ **do**
    Draw independently $x_0^{(i)} \sim p_{\text{data}}$, t_i uniformly from $1, \ldots, T$, and $\varepsilon^{(i)} \sim \mathcal{N}(0, I)$.
    $x^{(i)} \leftarrow \sqrt{\overline{\alpha}_{t_i}}\, x_0^{(i)} + \sqrt{1 - \overline{\alpha}_{t_i}}\, \varepsilon^{(i)}$
  end
  $\hat{\mathcal{L}}_{\text{noise}} \leftarrow \dfrac{1}{2B} \displaystyle\sum_{i=1}^{B} \left\| \varepsilon_\theta\big(x^{(i)}, t_i\big) - \varepsilon^{(i)} \right\|^2$
  Update θ using gradient $\nabla_\theta \hat{\mathcal{L}}_{\text{noise}}$ and learning rate η_j.
end
return θ

DDPM: generating samples by removing noise. Keep the trained network fixed. At each reverse step, its prediction sets the mean μ. Sample around that mean with variance β_t; the final step returns the mean without added noise.

Algorithm 2. DDPM sampling

Input: Fixed predictor ε_θ; schedule β_t, $\alpha_t = 1 - \beta_t$, $\overline{\alpha}_t = \prod_{j=1}^{t} \alpha_j$, for $t = 1, \ldots, T$.
Output: Generated sample x_0.

Draw $x_T \sim \mathcal{N}(0, I)$.
for $t = T, T-1, \ldots, 1$ **do**
  $\hat{\varepsilon} \leftarrow \varepsilon_\theta(x_t, t)$
  $\mu \leftarrow \dfrac{1}{\sqrt{\alpha_t}} \left(x_t - \dfrac{\beta_t}{\sqrt{1 - \overline{\alpha}_t}} \hat{\varepsilon} \right)$
  $z \leftarrow 0$
  if $t > 1$ **then**
    Draw $z \sim \mathcal{N}(0, I)$.
  end
  $x_{t-1} \leftarrow \mu + \sqrt{\beta_t} z$
end
return x_0

Conditional-mean prediction and the remaining training loss. The noisy point $x_t = (0, 0)$ could have come from any cluster. The predictor cannot identify which noise was added. A *conditional density* $p(u \mid y)$ describes an unknown u after observing y; the bar means "given". Its probability-weighted average is the conditional mean, $\mathbb{E}[u \mid y]$. For noise prediction, $u = \varepsilon$ and $y = x_t$, at a fixed t. At the origin, symmetry makes the mean noise zero.

Let a be any prediction made from y. Its average squared error separates into two terms:

$$\mathbb{E}[\|u - a\|^2 \mid y] = \mathbb{E}[\|u - \mathbb{E}[u \mid y]\|^2 \mid y] + \|\mathbb{E}[u \mid y] - a\|^2. \tag{10}$$

The first is the uncertainty left after observing y. The second measures how far the prediction is from the conditional mean. Only the second depends on a, so choosing $a = \mathbb{E}[u \mid y]$ minimizes the error. Noise-prediction training therefore aims at $\mathbb{E}[\varepsilon \mid x_t]$ at each level. The first term can remain positive even for the best predictor. It explains why the training loss need not reach zero.

With this exact prediction, the sampler's mean would be exact; a learned predictor approximates it. Treating the reverse distribution as Gaussian around the mean is a separate approximation, justified by small steps.

Why the mixture sampler must not clip to $[-1, 1]$. Replacing values below a lower bound by that bound, and values above an upper bound by that bound, is clipping. Clipping to $[-1, 1]$ would exclude this mixture's four centres, so we omit it.

KEY CONCEPTS
- **Target:** equal cluster populations, prescribed centres and coordinate spread 0.3.
- **Forward diffusion:** shrink and add Gaussian noise; the schedule sets its variance.
- **Learning:** squared noise error trains a time-conditioned predictor toward the conditional mean.

- **Generation:** DDPM uses the fixed predictor for reverse steps, with no added noise at the last step.

3 VERIFY

(1) **Forward diffusion turns four clusters into one Gaussian.**

```
Make a movie of forward diffusion showing how the modes merge. Follow the same
samples as the noise level increases, with a fixed coordinate scale. Print each
coordinate's mean and variance at t=1000.
```

The centres move toward the origin while each cloud broadens; by $t = 1000$ the four groups are indistinguishable. With at least 2000 samples, each coordinate's mean should lie within 0.1 of zero and its variance within 0.1 of one. Watch the reference forward process.

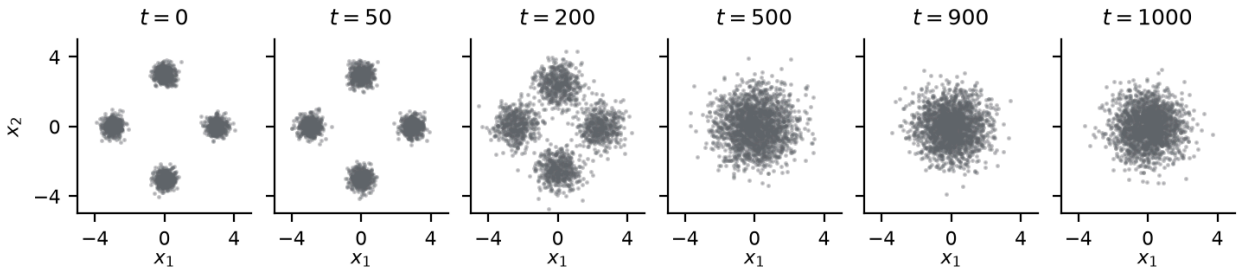


Figure 4: The forward end state is approximately standard Gaussian. The reference follows the same 2000 points through forward steps at six levels on one scale.

(2) **Training moves the generated endpoints onto the clusters.**

```
Make a movie of the 10 saved training stages. At each stage, show 10 DDPM
generation paths with the current predictor and the training-loss history so far.
Reuse the same initial Gaussian samples and reverse-step noise across stages.
```

Each path runs from $t = 1000$ to 0 with fixed weights; the training stage changes the weights. Shared starts and step noise isolate the effect of learning. At initialization the paths leave the plotted square, some reaching $|x| \approx 1500$; at the final stage all ten endpoints sit on the four clusters. In the reference most of the change happens in the first few hundred of its 25000 updates, while the loss falls. The loss then levels off, near 0.25 in the reference, rather than at zero. Part 1's conditional-mean identity explains this remaining loss: the added noise is still uncertain given the noisy point. Watch the reference training process.

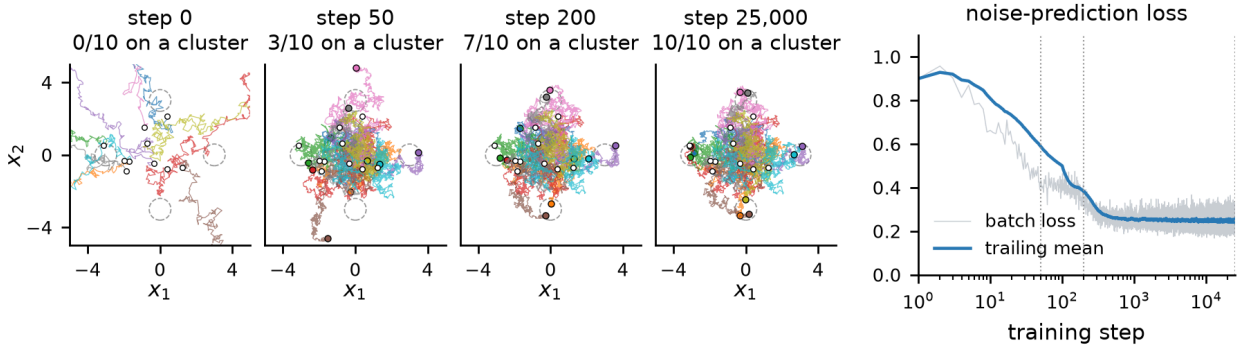


Figure 5: Endpoints move onto the four clusters as the loss falls. The same ten generation paths (white starts, coloured ends, dashed circles of radius 2σ around the centres) with the predictor saved at four of its ten training stages; dotted lines mark those stages on the loss history.

(3) **Generated samples recover the cluster populations, centres and widths.**

```
Generate 2000 samples with the trained predictor and 1000 DDPM steps. Make a movie
of the reverse process and compare the final distribution with the target. Plot the
samples and measure each mode's centre, population and per-coordinate spread.
```

Assign each generated point to its nearest target centre. Your populations should lie within 0.04 of 0.25, your spreads within 0.03 of 0.30 and your centres within 0.05 of the targets. The reference gives populations from 0.245 to 0.262 and spreads from 0.29 to 0.30. All three measurements are needed: correct populations can still hide shifted or broadened clusters. Watch the reference generation process.

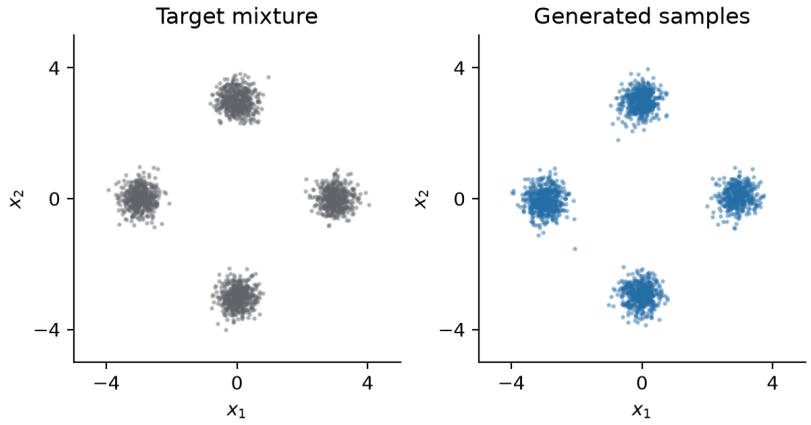


Figure 6: Generated samples recover the target's centres, equal populations and spread. The reference run's samples are plotted beside exact target samples.

Part 2: Deterministic sampling with DDIM

Goal: replace DDPM's random reverse step with a deterministic update that reuses the trained noise predictor and can jump between selected noise levels. Check that repeated runs from one starting noise agree exactly, and that the generated points still recover the four clusters.

① DO

```
Implement deterministic DDIM sampling with the trained noise predictor over a
selectable subset of noise levels. Also support the mixture's exact conditional-mean
noise prediction for comparison.
```

② UNDERSTAND

The deterministic DDIM update. Let $t' < t$ be the next level visited. Subtract the predicted noise and undo the shrinkage in Equation 7 to estimate the clean point $\hat{x}_0$. Combine this estimate with the predicted noise in the proportions for t'. The list of visited levels is the *grid*; fewer levels mean fewer network evaluations.

Algorithm 3. Deterministic DDIM sampling

Input: Fixed predictor ε_θ; schedule $\overline{\alpha}_t$ with $\overline{\alpha}_0 = 1$; decreasing levels $T = t_0 > t_1 > \ldots > t_S = 0$.
Output: Generated sample x_0.

Draw $x_T \sim \mathcal{N}(0, I)$.
for $k = 0, \ldots, S-1$ **do**
  Set $t = t_k$ and $t' = t_{k+1}$.
  $\hat{\varepsilon} \leftarrow \varepsilon_\theta(x_t, t)$
  $\hat{x}_0 \leftarrow \dfrac{x_t - \sqrt{1-\overline{\alpha}_t}\,\hat{\varepsilon}}{\sqrt{\overline{\alpha}_t}}$
  $x_{t'} \leftarrow \sqrt{\overline{\alpha}_{t'}}\,\hat{x}_0 + \sqrt{1-\overline{\alpha}_{t'}}\,\hat{\varepsilon}$
end
return x_0

DDIM's coefficients differ from DDPM's: deleting DDPM's random term alone does not give this update.

DDIM's clean estimate targets the conditional mean. A function estimating the clean point is a *denoiser*. The box obtains $\hat{x}_0$ by solving Equation 7 for x_0 and substituting the predicted noise. With the exact conditional-mean noise, this is *Tweedie*'s formula and returns $\mathbb{E}[x_0 \mid x_t]$. By Part 1's squared-error identity, that mean minimizes clean-point prediction error; it need not be the original training point.

Separating step error from predictor error. Larger jumps use fewer predictions but can distort the generated distribution. To separate this step error from network error, repeat the update with the mixture's exact noise prediction, $\varepsilon_{\text{exact}}(x_t, t) = \mathbb{E}[\varepsilon \mid x_t]$. Calculate the conditional mean within each Gaussian component, then average using the components' probabilities given x_t. This removes predictor error while keeping the same grid.

KEY CONCEPTS
- **DDIM:** reuse the noise predictor with a deterministic update between selected levels.
- **Denoiser:** the clean estimate targets the conditional mean; the grid controls step size.
- **Exact predictor:** removes network error so the remaining discrepancy tests the sampling steps.

③ VERIFY

(1) **DDIM's reverse step is deterministic; DDPM's is not.**

```
Run DDIM over 1000 levels twice from the same noise, and DDPM from the same
starting noise with two noise draws. Plot their trajectories and 2000 samples of
each beside exact samples. Print the largest difference between the two DDIM runs.
```

The two DDIM runs agree exactly; the reference prints 0. Across the 2000 shared starts, the two DDPM runs end in the same cluster only about a quarter of the time, as chance predicts for four clusters. Both samplers still reproduce the target: DDIM cluster fractions lie within 0.015 of 0.25, and both samplers' widths lie between 0.29 and 0.32.

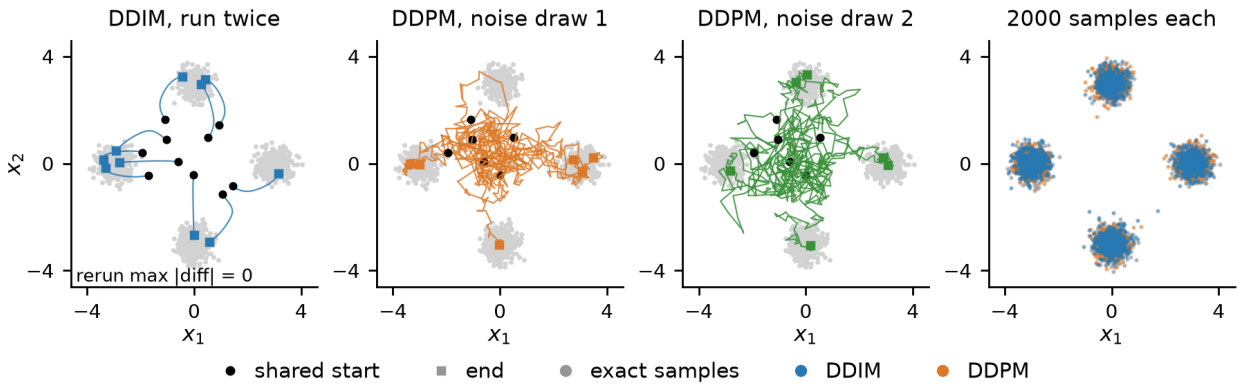


Figure 7: One starting noise gives one deterministic sample; the stochastic sampler does not, yet both reproduce the target distribution.

Part 3: Flow matching and the cost of fewer steps

Goal: train a velocity network on straight paths by flow matching, check it against the mixture's exact marginal velocity, and compare DDPM, DDIM and flow matching on one step grid. Measure how many steps each sampler needs to reach the target.

① DO

```
Train a velocity network on the same mixture by flow matching, with the path x_s=(1-
s)x_0+s z, z standard Gaussian, s uniform in [0,1], and target velocity z-x_0. Use the
noise predictor's architecture, trained from scratch, and a learning rate that decays
to zero. Sample by Euler steps from s=1 to s=0. Compute the exact marginal velocity of
the mixture for comparison.
```

```
Compare DDPM, DDIM and flow-matching samples with 1,2,5,10,20,50,100 steps, and 1000
levels for DDPM and DDIM. Fewer steps use evenly spaced levels. Use 4000 samples per
run. Record cluster fractions, centres and widths, and the energy distance to 4000
exact samples. Estimate its null scale from 20 pairs of independent exact sample sets.
```

2 UNDERSTAND

Constructing positions and velocity targets. Join a data point x_0 and an independent Gaussian point z by a straight path. Time s runs continuously from 0 to 1:

$$x_s = (1-s)x_0 + sz, \quad \frac{dx_s}{ds} = z - x_0. \tag{11}$$

Each pair supplies a position and a known velocity for training.

Predicting velocity with a new network. The network receives x_s and s and returns a velocity estimate $v_\theta(x_s, s)$. It reuses the noise predictor's architecture and output shape, but starts with newly initialized weights rather than the noise predictor's trained weights.

Training by squared velocity error. The target is the known path velocity $z - x_0$. The box averages squared errors over a batch of independent data, Gaussian draws and times s, with the same two-coordinate normalization as Part 1. Drawing s uniformly in $[0, 1]$ gives equal-length time intervals equal probability.

Algorithm 4. Train the velocity network

Input: Newly initialized weights θ; batch size B; J updates with learning rates η_j; optimizer; data distribution p_{data}.
Output: Trained weights θ.

for $j = 1, \ldots, J$ **do**
  for $i = 1, \ldots, B$ **do**
    Draw independently $x_0^{(i)} \sim p_{\text{data}}$, $z^{(i)} \sim \mathcal{N}(0, I)$, and s_i uniformly from $[0, 1]$.
    $x^{(i)} \leftarrow (1 - s_i)x_0^{(i)} + s_i z^{(i)}$
  end
  $\hat{\mathcal{L}}_{\text{flow}} \leftarrow \dfrac{1}{2B} \displaystyle\sum_{i=1}^{B} \left\| v_\theta\big(x^{(i)}, s_i\big) - \left(z^{(i)} - x_0^{(i)}\right) \right\|^2$
  Update θ using gradient $\nabla_\theta \hat{\mathcal{L}}_{\text{flow}}$ and learning rate η_j.
end
return θ

Euler sampling with the learned velocity. Generation starts from Gaussian noise at $s = 1$ and moves toward $s = 0$ with Euler steps, holding the velocity constant over one step of size $\Delta s > 0$:

$$x_{s - \Delta s} = x_s - \Delta s \, v_\theta(x_s, s). \tag{12}$$

The predicted velocity changes along the path, so finite steps introduce error.

Flow matching targets the average path velocity. Different pairs can pass through the same position at the same time with different velocities. The network cannot identify the pair from (x_s, s) alone. By Part 1's squared-error identity, the optimal prediction is their conditional average, the marginal velocity:

$$v(x, s) = \mathbb{E}[z - x_0 \mid x_s = x]. \tag{13}$$

Here "marginal" means averaging over the unobserved endpoints.

Why the average velocity transports the target distribution. A *velocity field* assigns a velocity $v(x, s)$ to each position and time. A point follows the *ordinary differential equation* (ODE)

$$\frac{dx_s}{ds} = v(x_s, s), \tag{14}$$

which specifies its rate of change; Euler steps approximate this motion. The *probability flux* is density times velocity; it describes probability crossing boundaries. Averaging the path velocities with their conditional probabilities preserves this flux. Thus the exact field transports the same distribution as the constructed paths. Network error and finite Euler steps can change the result, which is why we compare with the exact field and target.

Checking the network against the exact mixture velocity. Within each Gaussian component, the conditional average velocity can be calculated exactly. At (x, s), weight these component velocities by the probabilities that the components produced $x_s = x$; these probabilities are their densities there divided by the sum of those densities. This gives the exact mixture velocity $v(x, s)$. For N test points x_i at a fixed s, the *relative root-mean-square* (*RMS*) error is

$$\sqrt{\frac{\sum_{i=1}^{N} \|v_\theta(x_i, s) - v(x_i, s)\|^2}{\sum_{i=1}^{N} \|v(x_i, s)\|^2}}. \tag{15}$$

The numerator sums squared prediction errors; the denominator uses the exact field's squared lengths to set the scale. Square before averaging so opposite errors cannot cancel.

Skipping DDPM levels for the step-count comparison. The comparison also needs a random sampler on the coarser grid. For a jump from t to t', set $a = \overline{\alpha}_t / \overline{\alpha}_{t'}$. The forward jump shrinks by $\sqrt{a}$ and adds noise of variance $1 - a$. Use Part 1's reverse rule with these effective coefficients:

$$\mu_{t \to t'} = \frac{1}{\sqrt{a}} \left(x_t - \frac{1 - a}{\sqrt{1 - \overline{\alpha}_t}} \varepsilon_\theta(x_t, t) \right). \tag{16}$$

Add Gaussian noise of variance $1 - a$ when $t' > 0$, and none when $t' = 0$. For adjacent levels this is exactly Part 1's choice of variance β_t. The comparison uses evenly spaced levels before reaching zero.

Comparing sampler outputs with energy distance. The step-count plot needs a measure of discrepancy between the generated and target distributions. *Energy distance* uses distances between samples. Let X, X' be generated samples and Y, Y' exact target samples, with all four draws independent and finite expected lengths $\mathbb{E}\|X\|$ and $\mathbb{E}\|Y\|$. Then

$$D = \underbrace{2\mathbb{E}\|X - Y\|}_{\text{between distributions}} - \underbrace{\mathbb{E}\|X - X'\| + \mathbb{E}\|Y - Y'\|}_{\text{spread within each distribution}}. \tag{17}$$

Even matching distributions usually give $\mathbb{E}\|X - Y\| > 0$: independent draws need not coincide. The two subtracted terms remove this ordinary spread. If the distributions match, all three expected distances are equal and $D = 0$. To justify the converse and nonnegativity, in one dimension define the *cumulative probabilities* $F(r) = \Pr(X \le r)$ and $G(r) = \Pr(Y \le r)$. The distance $|X - Y|$ is the length of the interval between the two draws. A threshold r lies between them with probability $F(r)(1 - G(r)) + G(r)(1 - F(r))$. Integrating over r gives the expected distance. Apply this to all three terms of D and simplify:

$$D = 2 \int_{-\infty}^{\infty} (F(r) - G(r))^2 \, \mathrm{d}r. \tag{18}$$

For our two-dimensional samples, project each point onto a line. Euclidean length is $\frac{\pi}{2}$ times the average absolute projection length over uniform directions, so D is $\frac{\pi}{2}$ times the average of these nonnegative one-dimensional distances. Hence $D \ge 0$. If $D = 0$, the projected distributions agree in every direction: their energy distance is continuous in direction, so a positive value anywhere would make the average positive. All line projections together determine the joint distribution. Thus $D = 0$ exactly when the generated and target distributions match, justifying its use as a distribution discrepancy in the step-count plot. See Székely and Rizzo, Section 2.

Judging energy distance against sampling variation. Estimates from finite samples fluctuate; replacing the expectations by averages over distinct pairs can even give a negative estimate. Compare independent exact sample sets to measure this fluctuation, the *null scale*. Use their 95% *quantile*, the value below which 95% of those distances fall. A generated sample set within this scale shows no difference large enough for this test to distinguish at the chosen sample size.

Reading the uncertainty band on cluster fractions. Even exact draws do not put exactly a quarter of a finite sample in each cluster. Repeated sets of N independent points give slightly different fractions. The standard deviation of such an estimated quantity is its *standard error*. For a cluster with probability p, the fraction's standard error is $\sqrt{p(1 - p)/N}$. The reference plot shades two standard errors around $p = 0.25$. This is the *binomial* sampling variation, the variation in how many of N independent draws enter one chosen cluster.

For the conservation proof and conditional training construction, see the Flow Matching Guide, Sections 3.5 and 4.2–4.5. The guide runs time from noise to data; here data is at $s = 0$.

KEY CONCEPTS

- **Flow matching and marginal velocity:** regression on straight-line velocities learns their conditional mean, which Euler steps follow from noise to data.

- **Probability conservation:** the averaged path velocities carry the same probability flux as the constructed training paths.
- **Energy distance and null scale:** one number for two whole sample sets, judged against its spread between exact sets.

③ VERIFY

(1) **The learned velocity matches the exact marginal velocity.**

```
Plot the learned and exact velocity fields at s=0.1,0.5,0.9 on a grid over the
samples, and report their relative RMS difference at each s, the RMS of the
difference divided by the RMS of the exact velocity.
```

The arrows overlap; yours should differ by less than 0.1 at every s, and the reference gives 0.011, 0.017 and 0.058. The arrows show dx_s/ds for increasing s; generation runs against them. Near $s = 1$ the field approaches $v(x, 1) = x$, because the mixture's mean is zero.

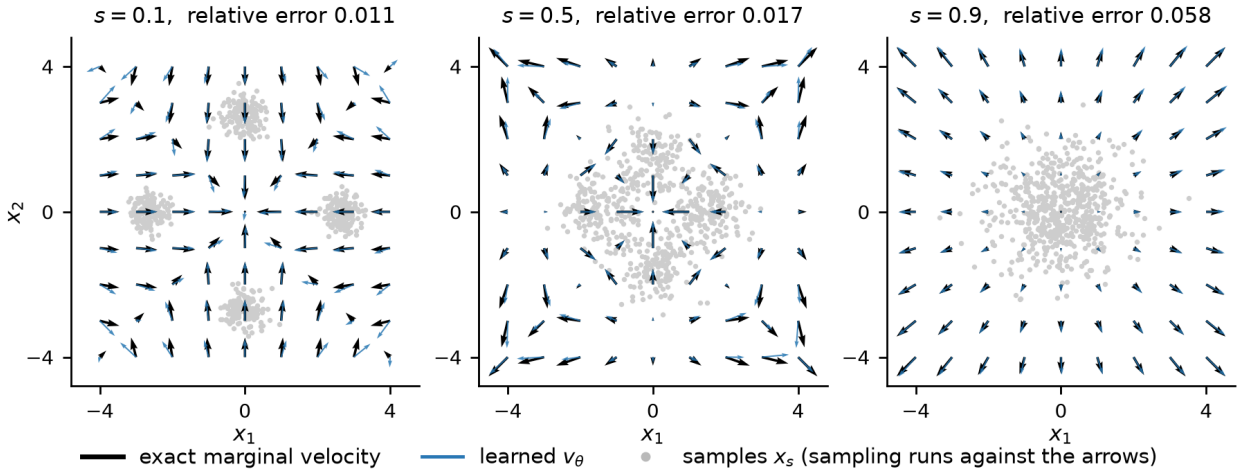


Figure 8: The learned field reproduces the mixture's marginal velocity, which is why Euler steps can follow it. Arrow lengths are clipped in the far field.

(2) **Flow matching and DDIM need tens of steps; DDPM needs more.**

```
Plot the energy distance, the largest cluster-fraction error and the largest width
error against the number of steps for the three samplers, with the null scale
shaded.
```

By 20 steps your flow sampler should reach an energy distance below 0.005 and widths within 0.03 of 0.30. The reference flow sampler reaches 0.0015 at 20 steps, at the edge of the null 95% quantile 0.0013; DDIM reaches 0.0038, and DDPM needs about 50 levels. One or two steps fail for every sampler, even with the exact noise prediction or velocity: the samples collapse toward the mean. On this mixture the gain over DDIM is modest.

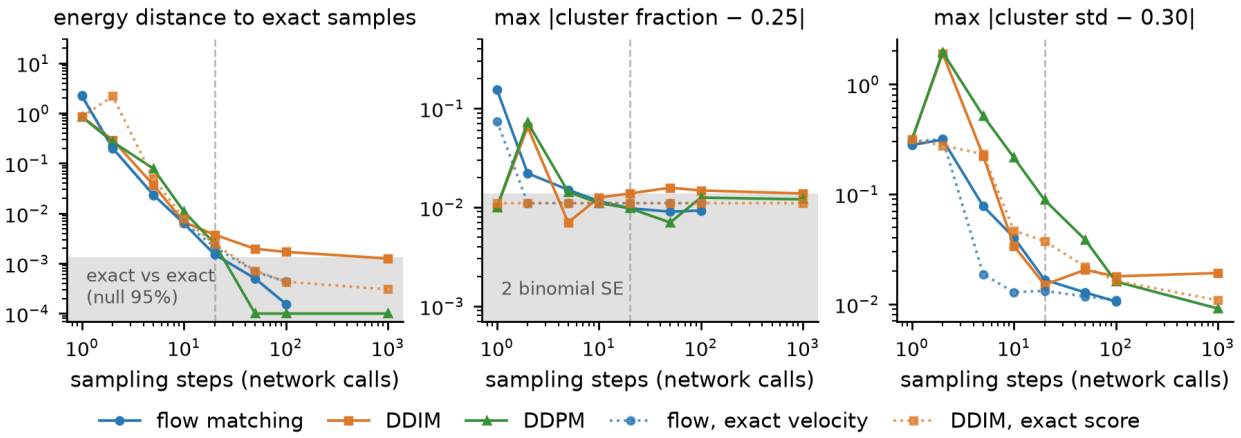


Figure 9: Flow matching and DDIM reach the target in tens of steps while DDPM needs about fifty. Shaded bands are the exact-versus-exact null scale and two binomial standard errors of a cluster fraction; energy distances below 10^{-4} are drawn at 10^{-4}. The dashed line marks 20 steps.

Part 4: The same predictor on photographs

Goal: load a predictor already trained on CIFAR-10 photographs and generate recognizable images with Part 1's DDPM update, without further training. Part 5 uses the same model to fill in missing pixels.

(1) DO

Download the CIFAR-10 noise predictor, already trained, from Hugging Face. Keep these files for Part 5.

```
uv run --with huggingface-hub==1.33.0 hf download google/ddpm-cifar10-32 \
  config.json diffusion_pytorch_model.safetensors \
  --revision 267b167dc01f0e4e61923ea244e8b988f84deb80 \
  --local-dir inputs/cifar-model
```

Generating four images takes roughly three minutes on the reference CPU.

```
Load the downloaded pretrained weights and configuration as a fixed noise predictor in
JAX. Match the network in this reference implementation:
https://github.com/huggingface/diffusers/tree/v0.40.0
```

```
Use the pretrained predictor with the same 1000-step DDPM sampler as on the mixture,
without clipping during sampling. Use float32 arrays and pass noise level t as model
index t-1. Keep the downloaded weights fixed.
```

(2) UNDERSTAND

Loading the predictor from weights and configuration. A pretrained network has already learned its weights. We use `google/ddpm-cifar10-32`, trained to predict noise in photographs. It is *unconditional*: it receives no requested object category or text. The weights file supplies its parameters; the *configuration* file specifies the architecture. The download's *revision* fixes the version of both. We keep those weights fixed throughout sampling.

Predicting one noise value per pixel and colour channel. Each pixel has red, green and blue values, its three *colour channels*. The image is an *array*, an indexed collection of numbers; its *shape* counts entries along each direction. For a batch of B images the predictor maps

$$\varepsilon_\theta : (x_t, t) \mapsto \hat{\varepsilon}, \quad x_t, \hat{\varepsilon} \in \mathbb{R}^{B\times 3\times 32\times 32}. \tag{19}$$

Channels come before height and width. The output estimates one noise value per pixel and channel, exactly the quantity Part 1's DDPM sampler needs.

Matching the checkpoint's indexing and colour range. Apply the same 1000-step DDPM algorithm to these arrays. Use *float32*, storing each number approximately with 32 binary digits. The checkpoint counts levels from zero: pass our

t as model *index* $t - 1$. Clean colours lie in $[-1, 1]$; for display map x to $\frac{x+1}{2}$ and clip to $[0, 1]$. Keep the sampling states unclipped.

Convolutions extract local image features. The image predictor uses *convolutions*: each slides a learned *filter* over the image. At each position it multiplies nearby values in all input channels by matching weights, sums the products and adds a learned bias. Each filter produces one *feature channel*, a grid of responses.

The U-Net combines coarse context with pixel detail. The *U-Net* [1] reduces the grid size, or spatial *resolution*, to gather wider context, then enlarges it to predict noise at every original pixel. Its down path reduces side lengths from 32 through 16 and 8 to 4; the up path returns to 32. *Skip connections* append saved down-path features to up-path features at matching resolutions, preserving detail. The final convolution returns three noise channels. The configuration fixes the layer dimensions and operations that the implementation must match. As in Part 1, *time features* computed from t are supplied throughout the network; here they are called the *time embedding*.

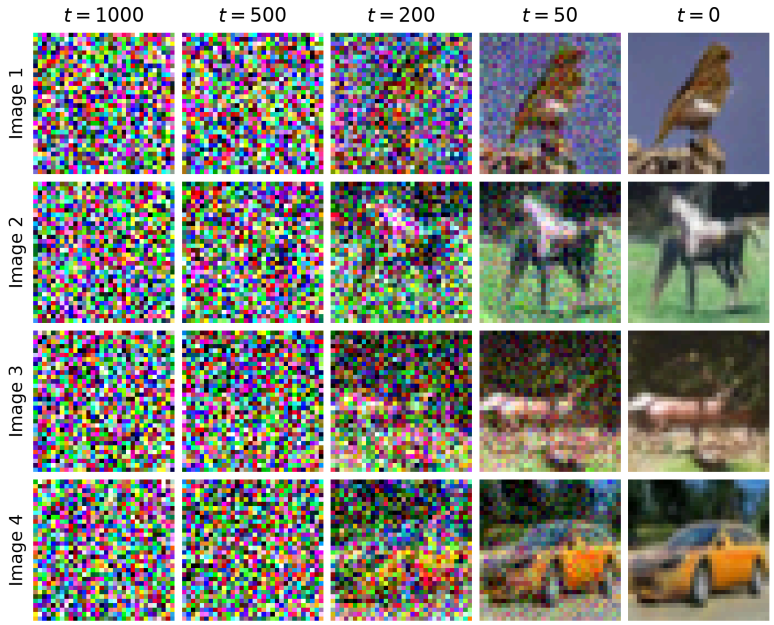
KEY CONCEPTS
- **Weights and configuration:** learned parameter values and the architecture needed to use them.
- **Noise prediction:** noisy image and noise level in, an image-shaped noise estimate out.
- **Generation:** Part 1′s DDPM update, unchanged, turns that estimate into an image.

3 VERIFY

(1) **The downloaded predictor generates recognizable photographs.**

```
Generate four images from Gaussian noise using the downloaded predictor and the
1000-step DDPM sampler. Visualize stages of the reverse process and the final
images. Compare them with real CIFAR-10 images using the Week 6 image-gallery
scripts.
```

Follow each noise image through the reverse process and compare the emerging colours, shapes and objects with the CIFAR-10 examples of Week 6′s Part 1. Structure appears late: at $t = 200$ the images are still mostly noise, and objects become recognizable only between $t = 200$ and $t = 50$. Explain which part of this process was learned before the download and which part your sampler computes now.



Pretrained noise predictor; the same DDPM update, all 1000 steps.
Recorded states; display clips to [−1, 1]. Native 32 × 32 pixels.

Figure 10: The downloaded weights and Part 1′s update produce recognizable objects without further training. Each row follows one sample; states are clipped only for display.

Part 5: Filling in missing pixels with the downloaded model

Goal: use the downloaded predictor to fill in a hidden square of a photograph, and check that every filled image keeps the visible pixels exactly. Repeating the process with different random choices should fill the hole with different, plausible content.

1 DO

Fetch the prepared CIFAR-10 images, stored in $[-1, 1]$, into your `week7/` directory. The file holds four test images.

```
mkdir -p inputs
curl -fL https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week7-images.npz \
  -o inputs/images.npz
```

```
With the pretrained CIFAR-10 predictor, inpaint images from observed pixels and a
mask, given the observations and the mask only. Implement RePaint with 50 reverse
jumps over levels 1000,980,...,20,0, two attempts per jump except the last, forward
noise between attempts, and known pixels replaced by forward-noised observations.
```

```
Use the four test images in inputs/images.npz. Hide rows and columns 6:26, all three
channels, and inpaint each image with RePaint using seeds 7201,7202,7203,7204. Record
each completion and its hidden-pixel mean squared error (MSE).
```

2 UNDERSTAND

Specifying visible and missing pixels with a mask. Filling the missing part of an image is inpainting. The filled image is a completion. The visible values are the *observations*; the sampler does not receive the hidden truth. A mask M with the image's shape specifies which values may change. It contains 1 at observed entries and 0 at hidden ones. Our mask hides a central 20×20 square in all three colour channels. Write Y for the observed image values where $M = 1$; set its unused entries to zero where $M = 0$.

Noising observations to match the sampling state. Before inserting observations into a state at level t', use Part 1's direct noising formula:

$$Y_{t'} = \sqrt{\overline{\alpha}_{t'}}\,Y + \sqrt{1 - \overline{\alpha}_{t'}}\,\eta, \quad \eta \sim \mathcal{N}(0, I). \tag{20}$$

These are the forward-noised observations.

Keeping observed pixels and proposing hidden pixels. RePaint is an inpainting sampler that replaces the observed pixels after every reverse step and can retry steps to improve the fit across the hole's boundary. Call the image suggested by a reverse step its *proposal*, $Z_{t'}$. Keep its hidden pixels, but replace its observed pixels with $Y_{t'}$. This combination is the *merge*:

$$X_{t'} = M \odot Y_{t'} + (1 - M) \odot Z_{t'}, \tag{21}$$

where $\odot$ multiplies matching entries. Both regions must be at the same noise level, which is why we use $Y_{t'}$. At $t' = 0$, $Y_{t'} = Y$, so the last merge preserves every observed value exactly.

Drawing RePaint's proposal across skipped levels. To reduce the number of network evaluations, RePaint visits $1000, 980, \ldots, 20, 0$. Each proposal jumps from t to $t' < t$, skipping levels as in Part 3's DDPM comparison. We use Part 2's Tweedie estimate $\hat{X}_0$, clipped to the clean image range $[-1, 1]$. Only this estimate is clipped, not the noisy state. Set $a = \frac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$ as in Part 3. If we treat the clean estimate as the original image, the Gaussian forward rule gives a conditional distribution for the lower-level state. Its mean is

$$\mu_{t \to t'} = \frac{\sqrt{\overline{\alpha}_{t'}}(1 - a)}{1 - \overline{\alpha}_t}\hat{X}_0 + \frac{\sqrt{a}(1 - \overline{\alpha}_{t'})}{1 - \overline{\alpha}_t}X_t. \tag{22}$$

The variance per coordinate is

$$v_{t \to t'} = \frac{(1 - \overline{\alpha}_{t'})(1 - a)}{1 - \overline{\alpha}_t}. \tag{23}$$

This differs from Part 3's choice $1 - a$ for the DDPM comparison. Draw the proposal $Z_{t'}$ around $\mu_{t \to t'}$ with this variance per coordinate, then apply the RePaint merge. At $t' = 0$ the variance is zero.

Repeating reverse jumps in the RePaint loop. Use two attempts for each jump except the final jump to zero, which is taken once. The cost is $49 \times 2 + 1 = 99$ network evaluations. Every Gaussian draw below is fresh, independent of earlier draws, and has the image's shape. As in Part 4, the predictor maps mathematical level t to checkpoint index $t - 1$.

Algorithm 5. RePaint inpainting

Input: Fixed predictor ε_θ; observations Y and mask M; schedule $\overline{\alpha}_t$ with $T = 1000$ and $\overline{\alpha}_0 = 1$.
Output: Completion X_0 with $M \odot X_0 = M \odot Y$.

Draw $X_T \sim \mathcal{N}(0, I)$ with the image's shape.
for $t = 1000, 980, \ldots, 20$ **do**
  Set $t' = t - 20$ and $a = \dfrac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$.
  Set $R = 2$ if $t' > 0$, otherwise $R = 1$.
  for $r = 1, \ldots, R$ **do**
    if $r > 1$ **then**
      Draw $f \sim \mathcal{N}(0, I)$.
      $X_t \leftarrow \sqrt{a} X_{t'} + \sqrt{1 - a} f$
    end
    $\hat{\varepsilon} \leftarrow \varepsilon_\theta(X_t, t)$
    $\hat{X}_0 \leftarrow \dfrac{X_t - \sqrt{1 - \overline{\alpha}_t}\hat{\varepsilon}}{\sqrt{\overline{\alpha}_t}}$
    Clip each entry of $\hat{X}_0$ to $[-1, 1]$.
    Compute $\mu_{t \to t'}$ and $v_{t \to t'}$ from Equation 22 and Equation 23 using the current X_t and $\hat{X}_0$.
    Draw independent $\xi, \eta \sim \mathcal{N}(0, I)$.
    $Z_{t'} \leftarrow \mu_{t \to t'} + \sqrt{v_{t \to t'}} \xi$
    $Y_{t'} \leftarrow \sqrt{\overline{\alpha}_{t'}} Y + \sqrt{1 - \overline{\alpha}_{t'}} \eta$
    $X_{t'} \leftarrow M \odot Y_{t'} + (1 - M) \odot Z_{t'}$
  end
end
return X_0

Why retrying lets hidden pixels respond to observations. The first proposal uses the state before the observations are merged. Adding forward noise to the merged state and repeating the jump lets the predictor respond to those observations when proposing the hidden pixels.

See Lugmayr et al., *RePaint*, Section 4.1, for the resampling method.

Measuring completion error and variation. The *hidden-pixel MSE* averages squared errors against the withheld truth over the hole's 400 pixels and three channels, 1200 values in total. Many images can agree with the observations, so a plausible completion can still have a large error. A *random seed* selects a sequence of random draws; different seeds produce different completions. The *pixelwise variance* across completions measures where they disagree. It measures variation, not error against the truth.

KEY CONCEPTS

- **Mask and inpainting:** the mask marks observed pixels; a completion keeps them and fills the rest.
- **Forward-noised observations:** the observed pixels, noised to the sampler's current level with Part 1's jump formula.
- **RePaint:** a coarse reverse sampler with the observed pixels overwritten after every jump; retry each jump after forward noising, except the final jump to zero.
- **Hidden-pixel MSE and the spread of completions:** error against the withheld truth; different seeds give different plausible completions where the observation says nothing.

3 VERIFY

(1) **Completions keep the observed pixels and disagree inside the hole.**

> For each test image, show the truth, the observation, the four completions and the pixelwise variance across them. Print each completion's hidden-pixel MSE and the largest change of an observed pixel.

Every observed pixel should change by less than 10^{-5}: RePaint overwrites them at the last level. Outside the hole the four completions are identical; inside it they differ. In the reference the hidden-pixel MSE averages 0.24 over images and seeds; on the ship below it ranges from 0.13 to 0.38 across seeds. Locate the disagreements on the variance map, and explain why a completion can look plausible and still be wrong.

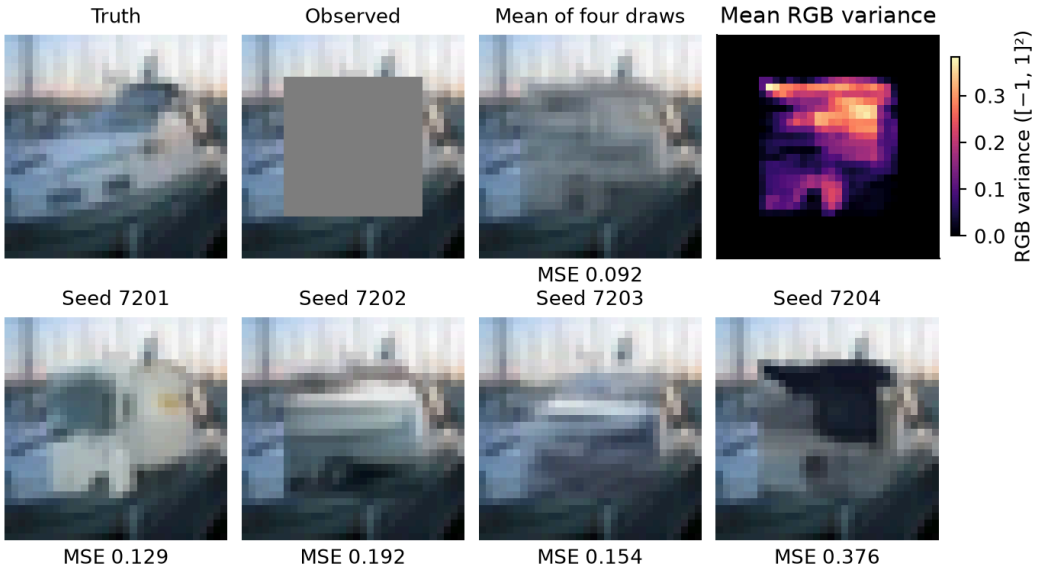


Figure 11: The observation leaves the hidden square undetermined, so each seed completes it differently. The reference ship image: truth, observation, the mean and pixelwise variance of four RePaint completions, and the four completions with their hidden-pixel MSE.

Save the results and prepare to show your work

> Update README.md with each evidence figure beside its producing command. Explain the transport comparison, the generated photographs and the inpainting result. Record training settings, environment setup and the pinned model download. Keep downloaded weights out of git. Push the source, environment and evidence.

Show the completed evidence at the start of Week 8. Nothing is submitted.

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.